

Computer Architecture and Concurrency, COMP0008 (A5U)

Main Summer Assessment 2023/24

Suitable for Cohorts: 2023/24, 2022/23, 2021/22

This assessment includes two parts. Part One has 2 sets of multiple choice questions and 1 MIPS programming question. Part Two includes 2 questions with 5 sub-questions each. Answer ALL the questions, and upload your answers on the UCL exam platform.

Important: You should submit typed answers (i.e., not handwritten) where possible, unless you hold a relevant SoRA. Please, submit your answers as a single PDF file.

Marks: Marks for all questions are indicated in square brackets.

For multiple choice questions (in Part One), you will be awarded full marks for each correct answer, partial marks for no answer, and zero marks for wrong answers. So, DO NOT GUESS. Marks for multiple choice questions are indicated by two values: the first value indicates the marks you will be awarded for a correct answer, and the second value corresponds to the marks you will be given in the case of no answer. For example, [(+5,+2) marks] next to a multiple choice question specifies that a correct answer to that question is worth 5 marks, and no answer is worth 2 marks.

Part ONE: Computer Architecture

Theoretical Questions

In your submission, specify your answers **using the following template**.

Part One: Computer Architecture

Q1a: < *your – answer* >

Q1b: < *your – answer* >

Q2a: < *your – answer* >

Q2b: < *your – answer* >

Q2c: < *your – answer* >

Note that your submission must include the sentence “Part One: Computer Architecture” before your answers and a comma after each of your answers. For each question, replace < *your – answer* > in the above template with A, B, C, or D, depending on which answer you consider correct. Any other text (including no text at all) followed by a comma will be considered as no answer to the corresponding question. **Note:** Exactly one option is correct; do not mark multiple options.

Question 1. In this question, we will scrutinize the different jump instructions.

Suppose that an instruction *I* is located at 0x400000. We are interested in what the PC value could be after the instruction is executed.

Q1a. Suppose that the instruction changed the PC value to 0x20400000.

[(+5,+1.5) marks]

- A) *I* can be the `j` instruction.
- B) *I* can be the `jal` instruction.
- C) *I* can be the `jr` instruction.
- D) *I* can be the `beq` instruction.

Q1b. Suppose that after executing the instruction *I*, the value of `$ra` changed.

[(+5,+1.5) marks]

- A) *I* can be the `j` instruction.
- B) *I* can be the `jal` instruction.
- C) *I* can be the `jr` instruction.
- D) *I* can be the `beq` instruction.

Q1c. Suppose that the instruction changed the PC value to `0x20400000` **and** the value of `$ra` changed.

[(+5,+1.5) marks]

- A) *I* can be the `j` instruction.
- B) *I* can be the `jal` instruction.
- C) *I* can be the `jr` instruction.
- D) *I* can be the `jalr` instruction.

Question 2. In the training of deep neural networks, practitioners often use floating point representations that are shorter than single-precision. To optimize the accuracy, an alternative to IEEE 754 half-precision (which uses 16 bits separated into a sign bit, 5 exponent bits, and 10 fraction bits) is called `bfloat16`. `bfloat16`, which uses a similar width but with 8 exponent bits and 7 fraction bits, is tailor-suited for such purposes. The exponent biases of IEEE 754 half-precision and `bfloat16` are 15 and 127, respectively.

Q2a. Compared with IEEE 754 half-precision, `bfloat16` is:

[(+5,+1.5) marks]

- A) Able to represent fewer normal numbers.
- B) Able to represent more integer values accurately.
- C) Able to represent more numbers in the range $[1, 2]$.
- D) Able to represent more numbers in the range $[80000, 90000]$.

Q2b. Consider representing π in both formats.

[(+5,+1.5) marks]

- A) In `bfloat16`, it will have an accurate representation with no error.
- B) In IEEE 754 half-precision it will have an accurate representation with no error.
- C) Both formats will have a representation error, but the error of `bfloat16` will be **lower**.
- D) Both formats will have a representation error, but the error of `bfloat16` will be **higher**.

MIPS Programming

In this section, you are asked to write MIPS code. You are allowed (and encouraged!) to use MARS to check your code.

In your submission, provide your answers **using the following template**. Notice that you are required to start Q3 on a new page.

< new – page >

Q3:

< your – code >

< new – page >

Here, *< your – code >* must start on a line separate from the question number and must work on the MARS simulator if copy-pasted (*without any manual formatting afterward*). Additionally, your code must not span across page boundaries. That is, you should start your code (preceded with the question number as above) on a new page, but it must not proceed to any additional pages. If needed, you may use smaller font size to fit more lines of code into a page. For example, try copy-pasting the following code into your MARS:

```
.data
msg: .asciiz "\nGood code!\n"
.text
li $v0, 4 # print string
la $a0, msg #pointer to the message
syscall
```

If you choose to answer with 'I don't know', you will be awarded with 5 marks (but none will be given for a wrong solution.)

Academic honesty: All code you submit must be written by you alone. Copying of code from student to student is a serious infraction; the course staff use extremely accurate plagiarism detection software to compare code submitted by all students and identify instances of copying of code; this software sees through attempted obfuscations such as renaming registers and reformatting, and compares the actual parse trees of the code. Rest assured that it is far more work to modify someone else's code to evade the plagiarism detector than to write code yourself!

Question 3. In this question, your submitted code must start with:

```
li $t0, 16384
```

Your task is to *efficiently* determine whether \$t0 is an integer power of 4, and output as follows: \$t1 should be set to 1 if \$t0 is a power of 4 and to 0 otherwise. For example, as $4^7 = 16384$, the code should terminate with \$t1=1 for the above input.

Your solution will be marked on outputting the correct value and on efficiency. We will use the MARS instruction counter to determine the number of instructions *executed* (and *not* lines of code), and marks will be deducted for inefficient solutions. We will test your code on multiple inputs and not just the one above.

[(+25,+5) marks]

Part TWO: Concurrency

This section contains 2 questions, each with 5 sub-questions. **Be brief** in your explanations: write one or two sentences per question.

In your submission, specify your answers **according to the template below**. For each question, replace *< your – answer >* in the following template with your actual answer to the question. Additional instructions on the format of your answers are provided in each question.

Part Two: Concurrency

Q4a Answer:

< your – answer >

Q4b Answer: *< your – answer >*

Q4b Explanation: *< your – answer >*

Q4c Answer: *< your – answer >*

Q4c Explanation: *< your – answer >*

Q4d Answer: *< your – answer >*

Q4d Explanation: *< your – answer >*

Q4e Answer:

< your – answer >

Q5a Answer:

< your – answer >

Q5b Answer: *< your – answer >*

Q5b Explanation: *< your – answer >*

Q5c Answer: *< your – answer >*

Q5c Explanation: *< your – answer >*

Q5d Answer: *< your – answer >*

Q5d Explanation: *< your – answer >*

Q5e Answer: *< your – answer >*

Q5e Explanation: *< your – answer >*

Question 4. Consider the following code.

```
1:  public class ValueHolder {
2:      private int x;
3:      private int y;
4:      private int z;
5:      public volatile String description;

6:      public ValueHolder() {
7:          x = 0;
8:          y = 0;
9:          z = 20;
10:         description = "Value Holder\nValues:";
11:     }

12:     public synchronized void set(int xVal,int yVal,int zVal){
13:         x = xVal;
14:         y = yVal;
15:         z = zVal;
16:     }

17:     public synchronized void add(int incX,int incY,int incZ){
18:         x += incX;
19:         y += incY;
20:         z += incZ;
21:     }

22:     public synchronized void setDescription(String newDesc){
23:         description = newDesc;
24:     }

25:     public void printDescription() {
26:         System.out.println(description);
27:         System.out.println("x: " + x + "\ny" + y + "\nz" + z);
28:     }

29:     public void print() {
30:         System.out.println(transform());
31:     }

32:     private int transform() {
33:         return x + y + z;
34:     }
35: }
```

```

36: public class Type1 extends Thread {
37:     private ValueHolder h;

38:     public Type1(ValueHolder holder) {
39:         h = holder;
40:     }

41:     public void run() {
42:         h.set(3,2,1);
43:         h.print();
44:     }
45: }

46: public class Type2 extends Thread {
47:     private ValueHolder h;

48:     public Type2(ValueHolder holder) {
49:         h = holder;
50:     }

51:     public void run() {
52:         h.add(1,1,1);
53:     }
54: }

55: public class Main {
56:     private static ValueHolder holder = new ValueHolder();

57:     public static void main(String[] args)
58:         throws InterruptedException{
59:         holder.setDescription("Coordinates Holder\nValues:");
60:         Thread t1 = new Type1(holder);
61:         Thread t2 = new Type2(holder);
62:         Thread t3 = new Type2(holder);
63:         t1.start();
64:         t2.start();
65:         t3.start();
66:         t1.join();
67:         holder.print();
68:         t2.join();
69:         t3.join();
70:     }
71: }

```


Answer the following questions. Remember the semantics of the `join()` method: as reported in the Java documentation, it allows one thread to wait for the completion of another. That is, calling `t.join()` within a thread causes the thread to pause execution until `t` terminates.

- a. Focus on the `run()` method in `Thread t1` (lines 41-44). For the instructions in this method, list the atomic actions to consider when computing possible interleavings according to the concurrency abstraction explained in the lectures. Then list the constraints (if any) between such atomic actions.

Report your answer in the following format.

Q4a Answer:

*<action1: one sentence describing the first atomic action>;
...
<actionM: one sentence describing the M-th atomic action>;
<empty line>
<constraint1: one sentence describing the first constraint (if any)>
...
<constraintN: one sentence describing the N-th constraint (if any)>
<empty line>*

[8 marks]

- b. Can value 8 be printed by Thread `t1`? If this is the case, describe an interleaving causing 8 to be printed; otherwise, motivate why 8 cannot be printed by `t1`.

Provide your answer in the below format.

Q4b Answer: *<yes or no>*

Q4b Explanation: *<one or two sentences>
<empty line>*

[3 marks]

- c. What is the maximum value that can be printed by Thread `t1`?

Provide your answer in the below format, motivating your answer.

Q4c Answer: *<number>*

Q4c Explanation: *<one or two sentences>
<empty line>*

[3 marks]

- d. Suppose that we swap the order in which the instructions `t1.join()` (line 66) and `holder.print()` (line 67) appear in the `main` method. After this code modification, `holder.print()` appears before `t1.join()`. Is the set of values that the `main` method potentially prints impacted in any way by this change?

Report your answer in the following format, detailing what is the exact impact of the instruction swap, or precisely motivating why there is no impact.

Q4d Answer: *<yes or no>*

Q4d Explanation: *<one or two sentences>*

<empty line>

[3 marks]

- e. Consider now the class `ValueHolder`, ignoring how it is accessed by the `Main` class. Discuss the visibility properties of every instance field of `ValueHolder` objects, following the below format.

Q4e Answer

Instance field: *<field name>*

Method or methods where the field is published: *<name of the methods>*

Is the field safely published? *<yes or no>*

Can the field cause threads to see partially constructed objects? *<yes or no>*

Can threads see stale values for this field? *<yes or no>*

Notes (if any): *<one or two sentences, if needed>*

<empty line>

[8 marks]

[Total for Question 4: 25 marks]

Question 5. Consider the following code snippet.

```
1:  class Market {
2:      private int size = 1;
3:      private Item[] buffer = new Item[size];
4:      private int count = 0;
5:      private int in = 0;
6:      private int out = 0;

7:      public synchronized void put(Item o)
8:          throws InterruptedException {
9:          while (count==size){
10:             wait();
11:         }
12:         boolean wasEmpty = count==0;
13:         buffer[in] = o;
14:         ++count;
15:         in=(in+1)%size;
16:         if (wasEmpty) {
17:             notifyAll();
18:         }
19:     }

20:     public synchronized Item get()
21:         throws InterruptedException {
22:         while (count==0){
23:             wait();
24:         }
25:         boolean wasFull = count==size;
26:         Item o = buffer[out];
27:         --count;
28:         out=(out+1)%size;
29:         if (wasFull) {
30:             notifyAll();
31:         }
32:         return o;
33:     }

34:     public synchronized void print() {
35:         System.out.println(Arrays.toString(buffer));
36:     }
37: }
```

```

38: class Buyer extends Thread {
39:     private String id;
40:     private Market market;

41:     public Buyer(String buyerId, Market m) {
42:         id = buyerId;
43:         market = m;
44:     }
45:
46:     public void run(){
47:         Item o = null;
48:         try {
49:             while(isWillingToBuy()) {
50:                 Thread.sleep(1000);
51:                 o = market.get();
52:                 if(!decideToBuy(o)) {
53:                     market.put(o);
54:                     synchronized(market){
55:                         market.notifyAll();
56:                     }
57:                 }
58:                 else{
59:                     break;
60:                 }
61:             }
62:         }
63:         catch (InterruptedException e){
64:             return;
65:         }
66:         showItem(o);
67:     }

68:     private boolean isWillingToBuy(){
69:         // omitted code, with no call to any method in
70:         // the Market class
71:     }
72:
73:     private boolean decideToBuy(Item o){
74:         // omitted code, with no call to any method in
75:         // the Market class
76:     }

77:     private void showItem(Item o){
78:         System.out.println("\nBuyer: " + id +
79:                             "\nBought item: " + o + "\n");
80:     }
81: }

```

```

82: class Seller extends Thread {
83:     private String id;
84:     private Market market;

85:     public Seller(String sellerId, Market m) {
86:         id = sellerId;
87:         market = m;
88:     }

89:     public void run(){
90:         while(isInBusiness()){
91:             Item o = produceItem();
92:             try {
93:                 Thread.sleep(1000);
94:                 if (o != null){
95:                     market.put(o);
96:                 }
97:             }
98:             catch (InterruptedException e){
99:                 return;
100:            }
101:        }
102:    }

103:    private boolean isInBusiness(){
104:        // omitted code, with no call to any method in
105:        // the Market class
106:    }

107:    private Item produceItem(){
108:        // omitted code, with no call to any method in
109:        // the Market class
110:    }
111: }

```

Consider a multi-threaded application that creates a single `Market` instance `M`, and then starts `X` `Seller` threads and `X` `Buyer` threads, with `X` bigger than 10. A reference to the `Market` instance `M` is passed to the constructors of all the created threads.

- a. Describe the liveness properties of the application, using the following format.

Q5a Answer

Liveness problem: *<deadlock, starvation, livelock or missed signals>*

Is the above code prone to this liveness problem? *<yes or no>*

Explanation: *<one or two sentences>*

<empty line>

[8 marks]

- b. Suppose that we change `while (count==0)` to `if (count==0)` inside the `get()` method of the `Market` class (line 22). Does this change impact the thread safety of the application?

Report your answer in the following format.

Q5b Answer: *<yes or no>*

Q5b Explanation: *<one or two sentences>*

<empty line>

[4 marks]

- c. Suppose that during the application's execution, we end up with a single `Seller` thread and several `Buyer` threads. All the other threads are already terminated. Can the `Seller` thread enter the `Market`'s condition queue and never be woken up?

Report your answer in the following format.

Q5c Answer: *<yes or no>*

Q5c Explanation: *<one or two sentences>*

<empty line>

[4 marks]

- d. Suppose that during the application's execution, we end up with a single `Buyer` thread and several `Seller` threads. All the other threads are already terminated. Can the `Buyer` thread enter the `Market`'s condition queue calling `market.get()` (line 51) and never exit the queue?

Report your answer in the following format.

Q5d Answer: *<yes or no>*

Q5d Explanation: *<one or two sentences>*

<empty line>

[4 marks]

- e. Suppose that we remove `market.notifyAll()` from inside the `run()` method of the `Buyer` class (line 55). Only methods in the `Market` class now include calls to `notifyAll()`.

After this code change, can any `Buyer` thread call the method `market.get()` (line 51) when there are exactly ten `Item` objects in the `Market`'s buffer?

Report your answer in the following format.

Q5e Answer: *<yes or no>*

Q5e Explanation: *<one or two sentences>*

<empty line>

[5 marks]

[Total for Question 5: 25 marks]